

Projeto A.H.M.A

Sistema integrado de monitoramento de hortas urbanas e catálogo botânico digital

A.H.M.A significa **Automação da Horta Mello Ayres**. O projeto combina uma aplicação web instalável, um serviço de autenticação e persistência baseado em Supabase, um catálogo botânico chamado **Plantopédia** e um protótipo de monitoramento de umidade do solo baseado em Arduino Uno.

Esta documentação descreve a arquitetura, o estado atual, o funcionamento, o modelo de dados, a organização do código, os limites conhecidos e o caminho necessário para concluir a telemetria entre o hardware e a aplicação web.

Estado geral: a autenticação, a navegação, a Plantopédia e a interface do dashboard de monitoramento estão implementadas. A coleta física existe como protótipo local no Arduino, mas a transmissão automática das leituras para o Supabase ainda precisa ser implementada.

Sumário

1. [Visão geral](#)
2. [Objetivos e escopo](#)
3. [Estado atual do projeto](#)
4. [Arquitetura do sistema](#)
5. [Tecnologias utilizadas](#)
6. [Fluxo funcional da aplicação](#)
7. [Hardware e protótipo IoT](#)
8. [Regras de leitura e classificação da umidade](#)
9. [Integração de telemetria ainda pendente](#)
10. [Modelo de dados](#)
11. [Segurança e controle de acesso](#)
12. [Organização do repositório](#)
13. [Descrição dos módulos do código](#)
14. [Progressive Web App e funcionamento offline](#)

15. Instalação e execução local
 16. Limitações e riscos técnicos
 17. Plano de conclusão
 18. Conclusão
 19. Referências
-

1. Visão geral

O Projeto A.H.M.A foi concebido para apoiar o cuidado de hortas urbanas por meio de duas funcionalidades principais:

1. **Monitoramento do solo:** leitura da umidade por sensores e apresentação dos dados associados a cada horta.
2. **Plantopédia:** catálogo de espécies cultivadas, com informações botânicas, orientações de cultivo, categorias e imagens.

A aplicação web está hospedada no GitHub Pages e utiliza JavaScript executado no navegador. O Supabase fornece autenticação, banco PostgreSQL e armazenamento de imagens. O Arduino Uno representa a camada física de medição.

O sistema foi organizado para separar a camada de apresentação, os serviços de dados e o hardware. Essa separação permite evoluir cada parte sem misturar responsabilidades: a interface pode ser alterada sem reescrever o firmware, e o firmware pode mudar sem alterar a estrutura visual da Plantopédia.

2. Objetivos e escopo

2.1 Objetivo geral

Desenvolver uma plataforma integrada para organizar informações de hortas, disponibilizar um catálogo botânico e acompanhar a umidade do solo por meio de sensores conectados a um microcontrolador.

2.2 Objetivos específicos

- Desenvolver uma aplicação web responsiva e instalável como PWA.
- Implementar autenticação de usuários usando Supabase Auth.
- Controlar o acesso às páginas internas da aplicação.
- Criar um catálogo de espécies com inclusão, listagem, filtro, detalhamento e exclusão.

- Permitir o armazenamento de fotos das plantas no Supabase Storage.
- Modelar hortas, sensores, espécies e leituras em banco relacional.
- Construir um protótipo de leitura de umidade com Arduino Uno e sensor HD-38.
- Exibir a umidade em percentual e classificá-la conforme faixas de referência.
- Preparar a aplicação para receber o histórico de telemetria no dashboard de cada horta.

2.3 Fora do escopo atual

A versão atual ainda não implementa, no firmware presente no repositório, uma conexão de rede capaz de enviar leituras ao Supabase. O Arduino realiza a leitura, exibe o resultado no LCD e escreve informações na porta serial. A transmissão para a aplicação web depende de uma etapa futura, que pode utilizar um módulo de comunicação ou um dispositivo intermediário.

Também não há, no código atual, um painel administrativo completo para cadastrar sensores e hortas pela interface. Esses registros precisam ser criados diretamente no banco ou por uma funcionalidade administrativa que ainda deverá ser desenvolvida.

3. Estado atual do projeto

Módulo	Estado	Situação observada
Autenticação	Funcional	O login e o cadastro utilizam Supabase Auth. O acesso às áreas internas é protegido por verificação de sessão e perfil.
Controle de acesso	Parcialmente funcional	O perfil possui <code>nivel_acesso</code> e <code>status</code> . A aplicação reconhece perfis administrativos, mas as políticas completas de autorização precisam ser garantidas no Supabase.
Hub e navegação	Funcional	O hub direciona para Monitoramento do Solo e Plantopédia. O menu lateral e o menu hambúrguer são reutilizados entre as páginas.
Plantopédia	Funcional	O catálogo consulta <code>especimes</code> , consulta categorias, permite

		inclusão e exclusão e usa Storage para fotos.
Dashboard de monitoramento	Implementado	As páginas das três hortas consultam <code>horta</code> , <code>sensor</code> e <code>informacao_sensor</code> , mostrando resumo, histórico, gráfico, tabela e estados vazios.
Registros de hortas e sensores	Pendente	As telas dependem de registros existentes no banco. Sem horta e sensor associados, não há telemetria para exibir.
Firmware Arduino	Protótipo funcional local	O sketch lê <code>A0</code> , converte a leitura para percentual, classifica a umidade e exibe os dados no LCD e no monitor serial.
Envio IoT para a aplicação	Não implementado	O sketch atual não faz requisição HTTP, MQTT ou outro envio de rede ao Supabase.
PWA	Funcional em estrutura	Existe <code>manifest.json</code> e service worker com cache de arquivos estáticos. O cache de dados e a operação offline têm limitações descritas adiante.

Durante a inspeção do ambiente do projeto, as tabelas `sensor`, `horta` e `informacao_sensor` não apresentavam registros públicos de leitura. Assim, o dashboard foi projetado para informar a ausência de configuração sem fabricar valores.

4. Arquitetura do sistema

O sistema pode ser dividido em três camadas principais.

4.1 Camada de captura: hardware IoT

Essa camada é representada pelo Arduino Uno, pelo módulo comparador HD-38 e pela sonda de umidade do solo. O sensor produz um sinal analógico, que é lido pela entrada `A0` do Arduino. O firmware converte o valor para uma escala percentual e classifica a condição do solo.

Na versão atual, essa camada funciona localmente. Ela ainda não possui um canal de comunicação com o Supabase.

4.2 Camada central: Supabase

O Supabase é utilizado como Backend as a Service. Ele fornece:

- **Supabase Auth:** criação de conta, login, logout e sessão baseada em token.
- **PostgreSQL:** armazenamento de usuários, categorias, espécies, hortas, sensores e leituras.
- **Supabase Storage:** armazenamento das fotos utilizadas na Plantopédia.
- **API REST gerada pelo banco:** acesso às tabelas a partir do cliente JavaScript.

A aplicação utiliza uma chave pública no navegador. Essa chave não deve ser tratada como segredo. A proteção dos dados deve ser implementada com políticas de Row Level Security (RLS), regras de Storage e validações no banco.

4.3 Camada de apresentação: cliente web

A camada cliente está em `Client/` e utiliza HTML, CSS e JavaScript modular. Ela é publicada no GitHub Pages e pode ser acessada por navegador ou instalada como PWA quando o navegador oferecer essa opção.

As páginas utilizam módulos ES diretamente no navegador. Não há framework de interface ou servidor próprio no repositório atual.

4.4 Fluxo geral de dados

O fluxo planejado é:

Plain Text

```
Sensor de umidade
  |
  v
Arduino Uno / dispositivo de comunicação
  |
  v
Tabela informacao_sensor no Supabase
  |
  v
Dashboard da horta no Client/monitoramento/
  |
  v
Usuário autenticado no navegador
```

No estado atual, a parte entre o Arduino e a tabela `informacao_sensor` ainda não está conectada. O dashboard já está preparado para consultar os registros quando eles existirem.

5. Tecnologias utilizadas

5.1 Frontend

- HTML5 para estrutura das páginas.
- CSS3 para layout, responsividade, cores, tipografia e componentes.
- JavaScript moderno com módulos ES.
- SVG gerado no navegador para o gráfico de histórico do monitoramento.
- Google Fonts importadas pelos estilos tipográficos.

5.2 Backend e serviços

- Supabase Auth para autenticação.
- PostgreSQL gerenciado pelo Supabase.
- Supabase Storage para fotos da Plantopédia.
- Cliente `@supabase/supabase-js` carregado por CDN.

5.3 PWA

- `manifest.json` para nome, ícones, cores, orientação e modo de exibição.
- `sw.js` para cache de arquivos estáticos e tentativa de fallback em falhas de rede.

5.4 Hardware

- Arduino Uno R3.
 - Módulo HD-38.
 - Sonda resistiva de umidade do solo.
 - Display LCD I2C 16x2.
 - Biblioteca `LiquidCrystal_I2C`.
-

6. Fluxo funcional da aplicação

6.1 Entrada e autenticação

A página `Client/index.html` apresenta duas abas: **Entrar** e **Cadastrar**.

No login, `Client/src/pages/login.js` coleta o e-mail e a senha e chama `signInWithPassword`. Depois da autenticação, a aplicação consulta a tabela `cadastro` para verificar se o usuário possui perfil e se a conta está ativa. Quando a validação é concluída, o navegador redireciona para `hub.html`.

No cadastro, a página valida o tamanho da senha e exige ao menos um número. A conta é criada pelo Supabase Auth. O cadastro estendido na tabela `cadastro` deve ser tratado por uma rotina de banco, trigger ou fluxo administrativo consistente, pois a autenticação e o perfil são entidades diferentes.

6.2 Proteção das páginas

As páginas internas chamam `protegerRota` antes de carregar funcionalidades protegidas. O `guard` busca a sessão atual e consulta o perfil relacionado ao usuário. Se a sessão não existir ou se o perfil estiver inativo, a aplicação encerra a sessão e retorna ao login.

O repositório possui dois arquivos com responsabilidade semelhante:

- `Client/src/auth/auth-guard.js`.
- `Client/src/config/auth-guard.js`.

O arquivo em `config` é o que aparece importado pelas páginas atuais de monitoramento. O arquivo em `auth` contém uma implementação anterior com diferenças de nome de tabela e regras de perfil. Essa duplicidade deve ser eliminada ou documentada para evitar manutenção em duas versões divergentes.

6.3 Hub

O arquivo `hub.html` é a tela principal após o login. Ele possui cartões para:

- Monitoramento do Solo.
- Plantopédia.

O cabeçalho apresenta a marca, o botão do menu e a ação de saída. O rodapé identifica o projeto e os desenvolvedores.

6.4 Seleção de hortas

O arquivo `sensores.html` apresenta três cartões de destino. Cada cartão aponta para uma página individual:

- `monitoramento/horta_soja.html`.
- `monitoramento/horta_dois.html`.

- `monitoramento/horta_tres.html` .

As páginas compartilham a mesma estrutura e variam por meio do identificador da horta definido no HTML.

6.5 Dashboard da horta

O módulo `Client/src/pages/monitoramento.js` executa as seguintes operações:

1. Verifica a sessão do usuário.
2. Identifica qual horta está sendo visualizada.
3. Consulta os registros da tabela `horta` .
4. Localiza o sensor associado.
5. Consulta as leituras em `informacao_sensor` .
6. Normaliza os valores entre `0` e `100` .
7. Mostra a última leitura e a condição correspondente.
8. Desenha o histórico em SVG.
9. Lista as leituras mais recentes em uma tabela.
10. Informa estados de ausência de horta, sensor ou leitura.

A tela não gera telemetria fictícia. Quando o banco está vazio, o usuário recebe uma mensagem indicando que a horta ainda não foi configurada.

7. Hardware e protótipo IoT

7.1 Componentes

O protótipo descrito pelo sketch é composto por:

1. **Arduino Uno R3:** microcontrolador responsável pela leitura e pelo processamento inicial.
2. **Módulo HD-38:** módulo comparador que recebe a sonda e disponibiliza uma saída analógica.
3. **Sonda resistiva:** elemento inserido no solo para variar o sinal conforme a umidade.
4. **LCD I2C 16x2:** display usado para apresentar a situação do solo e o percentual.

7.2 Ligações documentadas

Saída do HD-38	Arduino Uno	Função
VCC	5V	Alimentação do módulo
GND	GND	Referência elétrica comum
AO	A0	Leitura analógica da umidade

A ligação do display utiliza a biblioteca `LiquidCrystal_I2C` e o endereço configurado no sketch como `0x27`. A pinagem física do LCD não está detalhada no código e deve ser confirmada conforme o módulo utilizado.

7.3 Funcionamento do sketch

O arquivo `IoT/monitoramento_solo.ino` configura `A0` como entrada e inicializa a comunicação serial em `9600` baud. Também inicializa o LCD e sua iluminação.

A leitura é obtida por:

C++

```
moisterValue = 1023 - analogRead(A0);
```

Em seguida, o valor é convertido para percentual por:

C++

```
float calculaUmidade = moisterValue * 100.0 / 1023.0;
```

O firmware mostra o valor bruto e o percentual no monitor serial. No LCD, mostra uma mensagem de classificação e a umidade formatada.

O ciclo possui esperas de alguns segundos para exibição das mensagens e uma espera final de aproximadamente trinta segundos antes de uma nova leitura. Portanto, a frequência efetiva de atualização é baixa e não deve ser interpretada como monitoramento em tempo real sem ajustes adicionais.

7.4 Cuidados do sensor resistivo

Sondas resistivas podem sofrer corrosão e alteração de comportamento com o tempo, especialmente quando permanecem energizadas continuamente em solo úmido. Uma evolução recomendada é alimentar a sonda apenas durante a medição, calibrar o valor mínimo e máximo em condições reais e registrar a data da calibração.

8. Regras de leitura e classificação da umidade

O firmware atual divide a leitura em seis faixas. Essas faixas devem permanecer idênticas no firmware, no dashboard e na documentação para evitar interpretações diferentes.

Faixa	Classificação exibida pelo Arduino	Interpretação na aplicação
0% a 16%	Muito Seco	Condição crítica; avaliar irrigação.
17% a 33%	Seco	Umidade baixa; avaliar a necessidade de rega.
34% a 50%	Umidade Baixa	Monitorar a evolução da leitura.
51% a 67%	Ideal	Faixa considerada ideal pelo firmware.
68% a 83%	Úmido	Solo úmido; evitar recomendar rega automaticamente.
84% a 100%	Muito Úmido	Verificar excesso de água e drenagem.

Esses percentuais são uma escala relativa do sensor, não uma medida universal de água no solo. Para que representem uma condição agronômica confiável, é necessário calibrar o conjunto sensor-solo e definir os limites de acordo com as espécies cultivadas.

A classificação não deve ser apresentada como recomendação agronômica absoluta. Ela deve funcionar como indicador de apoio à decisão.

9. Integração de telemetria ainda pendente

9.1 Problema atual

O sketch presente no repositório mede e exibe o valor, mas não possui código de rede. O Arduino Uno sozinho não oferece conectividade Wi-Fi ou Ethernet integrada. Dessa forma, não há atualmente uma rota automática entre a leitura física e a tabela `informacao_sensor`.

9.2 Alternativas de implementação

A integração pode ser concluída por uma destas abordagens:

- Substituir ou complementar o Arduino Uno com um ESP8266 ou ESP32.
- Utilizar um módulo Wi-Fi ou Ethernet compatível.
- Usar um computador intermediário que leia a porta serial e envie os dados à API do Supabase.
- Adotar um broker MQTT e um serviço intermediário responsável por persistir os dados.

A escolha deve considerar alimentação elétrica, alcance da rede, segurança, custo, disponibilidade de manutenção e quantidade de sensores.

9.3 Fluxo recomendado

1. O sensor realiza uma leitura.
2. O microcontrolador calcula a umidade.
3. O dispositivo acrescenta um identificador de sensor e um horário.
4. O dispositivo ou serviço intermediário envia a leitura para uma função segura ou endpoint de ingestão.
5. O backend valida o sensor e o intervalo permitido.
6. O backend insere o registro em `informacao_sensor`.
7. O dashboard consulta e exibe o histórico.

Não é recomendável colocar uma chave privilegiada do Supabase no firmware ou no código público do navegador. A ingestão deve usar uma credencial limitada, uma Edge Function ou outro mecanismo com validação e controle de abuso.

9.4 Validações necessárias

A ingestão deve rejeitar leituras que:

- Não contenham sensor válido.
- Tenham umidade menor que `0` ou maior que `100`.
- Possuam horário inválido ou muito distante do horário atual.
- Sejam enviadas em frequência incompatível com o dispositivo.
- Tentem gravar em uma horta ou sensor sem autorização.

10. Modelo de dados

10.1 Entidade `cadastro`

Armazena o perfil complementar do usuário autenticado. O identificador deve corresponder ao usuário criado em `auth.users`.

Campo	Tipo	Descrição
<code>id_conta</code>	<code>uuid</code>	Chave primária e referência para <code>auth.users(id)</code> .
<code>email</code>	<code>text</code>	E-mail do perfil. Deve ser único.
<code>nivel_acesso</code>	<code>text</code>	Perfil de acesso, como <code>USER</code> , <code>AGRO</code> ou <code>TI</code> , conforme a regra adotada.
<code>data_criacao</code>	<code>date</code>	Data de criação do perfil.
<code>status</code>	<code>boolean</code>	Indica se a conta está ativa.

10.2 Entidade `categoria_especimes`

Armazena categorias do catálogo botânico, como Hortalça, Erva / Tempero, Fruta, Flor e Outra.

Campo	Tipo	Descrição
<code>id_categoria</code>	<code>bigint</code>	Identificador gerado automaticamente.
<code>nome_categoria</code>	<code>text</code>	Nome único da categoria.

10.3 Entidade `especimes`

Armazena as plantas exibidas na Plantopédia.

Campo	Tipo	Descrição
<code>id_planta</code>	<code>bigint</code>	Identificador gerado automaticamente.
<code>nome_popular</code>	<code>text</code>	Nome comum da planta. Obrigatório.
<code>nome_cientifico</code>	<code>text</code>	Nome científico, quando disponível.

informacoes_adicionais	text	Informações de cultivo e observações.
foto_url	text	URL pública da imagem no Storage.
id_categoria	bigint	Referência para categoria_especimes .

10.4 Entidade sensor

Representa cada dispositivo de medição identificado pela aplicação.

Campo	Tipo	Descrição
id_sensor	bigint	Identificador do sensor.
nome_sensor	text	Nome descritivo do dispositivo.

10.5 Entidade horta

Representa cada unidade de cultivo monitorada.

Campo	Tipo	Descrição
id_horta	bigint	Identificador da horta.
nome_horta	text	Nome exibido ao usuário.
id_sensor	bigint	Sensor associado. Pode ser nulo enquanto a horta não estiver configurada.

10.6 Entidade associativa horta_planta

Representa a relação muitos-para-muitos entre hortas e espécies. Uma horta pode conter várias espécies, e uma espécie pode estar presente em várias hortas.

Campo	Tipo	Descrição
id_planta	bigint	Referência para especimes .

id_horta	bigint	Referência para horta .
----------	--------	-------------------------

A combinação dos dois campos deve ser a chave primária composta.

10.7 Entidade informacao_sensor

Armazena o histórico de telemetria.

Campo	Tipo	Descrição
id_resposta	uuid	Identificador único da leitura.
umidade	numeric	Percentual de umidade, idealmente entre 0 e 100 .
data_hora	timestampz	Data e hora da leitura.
id_sensor	bigint	Sensor responsável pela leitura.

O campo data_hora deve ser armazenado com fuso horário. A aplicação deve converter o valor para o fuso local apenas na apresentação.

10.8 Relações

Plain Text

```
auth.users 1—1 cadastro
categoria_especimes 1—N especimes
sensor 1—N horta
horta N—N especimes por meio de horta_planta
sensor 1—N informacao_sensor
```

A relação sensor 1—N horta representa o modelo atual do banco. Caso um sensor não possa ser compartilhado entre hortas, essa regra precisa ser aplicada com uma restrição adicional ou validada no serviço de cadastro.

11. Segurança e controle de acesso

11.1 Autenticação não é autorização

O Supabase Auth confirma a identidade do usuário, mas não define sozinho quais tabelas ou ações ele pode utilizar. A autorização depende da combinação entre o perfil em `cadastro`, a lógica da aplicação e as políticas de Row Level Security.

11.2 Regras recomendadas

- Habilitar RLS em todas as tabelas expostas pela API.
- Permitir que usuários autenticados leiam somente os dados destinados ao seu perfil, quando houver separação por usuário.
- Permitir escrita na Plantopédia somente aos perfis autorizados.
- Permitir inserção de telemetria somente por uma função ou credencial própria do dispositivo.
- Restringir exclusão de plantas e imagens a perfis administrativos ou autorizados.
- Validar `status` antes de permitir acesso às páginas internas.
- Não utilizar a `service_role` key no frontend, no GitHub Pages ou no firmware público.
- Aplicar políticas também ao bucket `plantas-fotos`.

11.3 Informações públicas no frontend

A URL do projeto Supabase e a chave pública podem aparecer no código do navegador. Isso é esperado para aplicações que utilizam a chave anon. O risco não está na exposição da chave pública, mas em deixar tabelas, Storage ou funções sem políticas adequadas.

11.4 Inconsistências a resolver

O repositório possui implementações antigas e atuais de autenticação. Em especial, `Client/src/services/authService.js` importa caminhos e nomes diferentes dos usados pelo fluxo ativo. O projeto deve escolher uma única implementação, corrigir imports obsoletos e remover arquivos não utilizados para evitar que uma manutenção futura reative um fluxo incompatível.

12. Organização do repositório

Plain Text

```
Projeto_AHMA/  
├── Client/  
│   ├── assets/  
│   │   ├── css/  
│   │   └── base/
```

```
| | | | components/
| | | |   pages/
| | | | |
| | | | | icons/
| | | | |   images/
| | | | |
| | | | | monitoramento/
| | | | |   horta_soja.html
| | | | |   horta_dois.html
| | | | |   horta_tres.html
| | | | |
| | | | | src/
| | | | |   auth/
| | | | |   components/
| | | | |   config/
| | | | |   pages/
| | | | |   services/
| | | | |
| | | | | hub.html
| | | | | index.html
| | | | | manifest.json
| | | | | plantopedia.html
| | | | | sensores.html
| | | | | sw.js
| | | | |
| | | | | IoT/
| | | | |   monitoramento_solo.ino
| | | | |
| | | | | LICENSE
| | | | | README.md
| | | | | documentação.md
```

12.1 Client/assets/css/base

- `reset.css` : remove margens e espaçamentos padrão e define o comportamento básico do layout.
- `typography.css` : importa fontes e estabelece a tipografia global.
- `variables.css` : concentra cores, sombras, raios de borda, espaçamentos e valores reutilizados.

12.2 Client/assets/css/components

- `content-boxes.css` : define cartões, grades e blocos de conteúdo usados no hub e na seleção de hortas.
- `header.css` : estiliza o cabeçalho, a marca e a ação de saída ou retorno.
- `footer.css` : define o rodapé comum.
- `nav-hamburger.css` : define a barra lateral, o overlay e o botão do menu.

12.3 Client/assets/css/pages

Contém os estilos específicos do login, da Plantopédia, da seleção de sensores e do dashboard de monitoramento.

12.4 Client/src

Contém a lógica JavaScript modular da aplicação. A divisão em `auth`, `config`, `components`, `pages` e `services` procura evitar que cada página implemente novamente as mesmas operações.

12.5 IoT

Contém o código destinado ao Arduino. O sketch atual é independente do frontend e ainda não possui uma biblioteca de transmissão para o Supabase.

13. Descrição dos módulos do código

13.1 Autenticação

`login.js` controla os formulários, as abas, as mensagens de erro, a visibilidade das senhas e as chamadas ao Supabase Auth.

`config/auth-guard.js` valida a sessão, busca o perfil em `cadastro`, verifica o status da conta e impede o acesso a páginas protegidas.

`services/logout.js` encerra a sessão e direciona o usuário de volta à tela de login.

13.2 Plantopédia

`plant-service.js` concentra as operações da tabela `especimes`, das categorias e do Storage. O serviço também implementa um cache local leve para acelerar a exibição inicial do catálogo.

`plantopedia.js` controla a interface, a busca, os filtros, os modais e os formulários. O serviço e a interface devem permanecer separados: a página não deve duplicar diretamente as consultas ao banco.

13.3 Monitoramento

`monitoramento.js` concentra a consulta das hortas, dos sensores e das leituras. Ele transforma os registros brutos em informações de apresentação e mantém os estados de carregamento, vazio e erro.

`monitoramento.css` contém os estilos do dashboard, incluindo cartões de resumo, gráfico, faixas de referência, tabela e comportamento responsivo.

13.4 Componentes de interface

`menu-hamburger.js` alterna as classes do menu lateral e do overlay. O componente depende da existência dos elementos `BtnMenu`, `sidebar`, `overlayMenu` e `conteudo-principal`.

Como esse script é carregado em várias páginas, alterações nele devem ser testadas em login, hub, Plantopédia, sensores e páginas de horta.

14. Progressive Web App e funcionamento offline

O `manifest.json` define os ícones em 192 e 512 pixels, versões maskable, cor de fundo, cor do tema, orientação vertical e modo `standalone`.

O `sw.js` realiza cache dos arquivos estáticos. Para arquivos do Supabase, tenta primeiro a rede e recorre ao cache quando a rede falha. Essa estratégia é útil para leituras já consultadas, mas não transforma a aplicação em um sistema completamente offline.

As seguintes operações dependem de conectividade:

- Login e logout.
- Cadastro de usuário.
- Inclusão e exclusão de plantas.
- Upload e exclusão de imagens.
- Envio de telemetria.
- Atualização de leituras não presentes no cache.

Uma evolução possível é utilizar IndexedDB para armazenar leituras e alterações pendentes, com uma fila de sincronização quando a rede retornar. O `localStorage` atual é adequado para um cache pequeno de catálogo, mas não para um histórico grande de sensores.

15. Instalação e execução local

O projeto atual é estático e não possui um processo de build obrigatório para executar o frontend.

Bash

```
git clone https://github.com/FelipeCorreia-TI/Projeto_AHMA.git
cd Projeto_AHMA/Client
python3 -m http.server 8080
```

Depois, acessar:

Plain Text

```
http://localhost:8080/
```

Também é possível utilizar:

Bash

```
npx serve .
```

O acesso ao Supabase depende da configuração existente em `Client/src/config/supabase.js`. Para utilizar outro projeto Supabase, devem ser alterados o endereço do projeto, a chave pública, as tabelas, as políticas e o bucket de Storage.

15.1 Bibliotecas do Arduino

Para compilar o sketch, a Arduino IDE precisa ter instalada a biblioteca compatível com `LiquidCrystal_I2C`. A ligação do LCD deve usar o endereço correto do módulo. O código atual assume `0x27`, mas alguns displays utilizam outro endereço.

15.2 Verificações após a instalação

- Confirmar que o login funciona.
- Confirmar que a conta possui registro em `cadastro`.
- Confirmar que o bucket `plantas-fotos` existe e possui políticas adequadas.
- Confirmar que existem categorias para a Plantopédia.
- Confirmar que as hortas e os sensores necessários foram cadastrados.
- Confirmar que o navegador consegue carregar os módulos ES.
- Confirmar que o service worker foi atualizado após mudanças no cache.

16. Limitações e riscos técnicos

16.1 Telemetria não concluída

A principal limitação é a ausência de comunicação entre o Arduino e o Supabase. O dashboard está pronto para leitura, mas não recebe automaticamente valores do dispositivo físico.

16.2 Calibração relativa

O cálculo 0% a 100% é uma normalização da faixa analógica. Ele não representa, por si só, uma porcentagem universal de água no solo. A calibração deve ser realizada com o sensor instalado no solo real.

16.3 Dependência do cliente público

A aplicação acessa o Supabase diretamente pelo navegador. Isso simplifica a arquitetura, mas exige políticas RLS rigorosas e validação no backend para qualquer operação sensível.

16.4 Código legado e duplicidade

Existem arquivos com nomes e imports que indicam versões anteriores da arquitetura, especialmente na autenticação e nos serviços. A manutenção deve diferenciar arquivos ativos de arquivos legados e remover dependências quebradas.

16.5 Banco ainda pouco operacionalizado

O modelo de dados está documentado, mas o funcionamento completo depende de registros reais de categorias, hortas, sensores, perfis e leituras. É necessário definir o processo de carga inicial e o responsável por cada tipo de cadastro.

16.6 Frequência de medição

O sketch utiliza vários `delay`, o que bloqueia a execução e torna o intervalo de leitura pouco flexível. Uma versão de produção deveria usar temporização não bloqueante com `millis()` e definir uma frequência configurável.

16.7 Validação elétrica e ambiental

O projeto deve validar alimentação, proteção contra umidade, estabilidade do sensor e segurança das conexões antes de instalar o conjunto permanentemente em uma horta.

17. Plano de conclusão

Fase 1 — Limpeza e padronização

- Escolher um único `auth-guard.js`.
- Remover ou corrigir o `authService.js` legado.
- Padronizar os nomes `cadastro`, `supabase.js` e os campos do perfil.
- Documentar quais arquivos são fonte e quais são versões minificadas.

Fase 2 — Segurança do Supabase

- Habilitar RLS em todas as tabelas.
- Criar políticas para leitura, inclusão, atualização e exclusão.
- Criar políticas do bucket `plantas-fotos`.
- Separar permissões de usuário comum, AGRO e TI.
- Criar uma estratégia segura para inserção de telemetria.

Fase 3 — Cadastro inicial

- Criar categorias padrão.
- Criar as três hortas ou definir seus nomes oficiais.
- Cadastrar cada sensor e seu identificador físico.
- Associar sensores às hortas.
- Associar espécies às hortas por `horta_planta`.

Fase 4 — Comunicação IoT

- Escolher Wi-Fi, Ethernet, ESP32, ESP8266, gateway serial ou MQTT.
- Definir o formato da mensagem de telemetria.
- Implementar autenticação limitada do dispositivo.
- Criar endpoint ou Edge Function de ingestão.
- Validar faixa, horário, sensor e frequência.
- Registrar logs de falhas de comunicação.

Fase 5 — Validação do dashboard

- Inserir leituras de teste controladas.
- Conferir a ordenação temporal.
- Comparar o valor exibido com o valor inserido.
- Testar estados sem sensor e sem leitura.
- Testar telas em celular e desktop.
- Verificar atualização do service worker.

Fase 6 — Operação e manutenção

- Definir periodicidade de calibração.

- Monitorar sensores sem comunicação.
 - Implementar alertas para umidade crítica, se necessário.
 - Criar rotina de limpeza ou retenção do histórico.
 - Registrar alterações de esquema e políticas do banco.
-

18. Conclusão

O Projeto A.H.M.A possui uma base coerente para uma plataforma de apoio ao cuidado de hortas. A aplicação cliente já separa autenticação, navegação, catálogo botânico, monitoramento e estilos reutilizáveis. O Supabase oferece os serviços necessários para autenticação, dados relacionais e armazenamento de imagens. O Arduino realiza a primeira etapa do sensoriamento e classifica a umidade localmente.

A etapa que falta para completar o ciclo principal é a transmissão confiável entre o dispositivo e o backend. Essa integração precisa ser acompanhada de políticas de segurança, cadastro inicial dos sensores e validação da calibração. Sem esses itens, o dashboard funciona como interface preparada e como visualizador de dados, mas não como um sistema IoT completo em produção.

Quando a telemetria for implementada, o sistema passará a cobrir o fluxo completo: medição no solo, processamento no dispositivo, persistência no Supabase, consulta autenticada e apresentação no PWA.

19. Referências

- [1] Arduino Uno Rev3 — documentação oficial
 - [2] Supabase Documentation — Database, Auth, Storage e Row Level Security
 - [3] MDN Web Docs — Progressive Web Apps
 - [4] MDN Web Docs — Service Worker API
 - [5] Repositório oficial do Projeto A.H.M.A
 - [6] Aplicação web publicada do Projeto A.H.M.A
 - [7] ABNT NBR 14724 — Informação e documentação: trabalhos acadêmicos
-

Documento revisado com base no código, no modelo de dados, no sketch IoT, no README, no site publicado e na documentação fornecida do Projeto A.H.M.A.